

Lab Practice Problem 11B

Filename: No File Submission

In this activity, you will work with tries by performing insertion and deletion operations and implementing trie-related functionality in code. The goal of this exercise is to strengthen your understanding of trie structures and their behavior.

Important: Do not run any program or use code to compute the results for this activity. The purpose of this exercise is to practice tracing the algorithm manually.

Trie Insertion

Draw the trie obtained by inserting the following words, in the given order, into an initially empty trie. After constructing the trie, determine the total number of nodes, including the root.

Hint: Every trie includes a root node, even when it stores no words.

cat, car, cart, carbon, dog, dove, dot, apple, app, ape

Trie Deletion

Using the trie constructed in the previous section, delete the following words in the specified order. After performing all deletions, determine the total number of nodes remaining in the trie, including the root.

Hint: Every trie includes a root node, even when it stores no words.

app, apple, cat

Coding Exercise 1

Complete the function that returns the total number of words stored in a trie. Assume the trie contains only lowercase English letters (26 characters).

Hint: This function can be naturally implemented using recursion.

```
int count(TrieNode *node) {
```

```
}
```

Coding Exercise 2

Complete the function that counts how many words in the trie begin with a given prefix. The function takes two parameters: the trie and the prefix. Assume the trie contains only lowercase English letters (26 characters). Also, the prefix is non-null and properly null-terminated.

Hint: Recall that a C string is null-terminated and represented as an array of characters. You may use pointer arithmetic to traverse the prefix. This problem is similar to Coding Exercise 1.

```
int count_starts_with(TrieNode *node, char *prefix) {
```

```
}
```

Code Implementation

Trie Implementation

```
typedef struct TrieNode_s {  
    struct TrieNode_s *children[26];  
    int is_end_of_word;  
} TrieNode;
```